How our coding assistant works

Language models, CEU AI and FirstRung

Coding 2: Python for Data Analysis

A request and a reply

Your request (the prompt)

Create analysis.py. Load data/raw/hotels_course.csv
and print the number of rows and columns.

An illustrative reply

```
import pandas as pd
hotels = pd.read_csv("data/raw/hotels_course.csv")
print(hotels.shape)
```

The reply contains code as text. Python runs that code when you execute the file.

Where the reply comes from

A large language model (LLM) generates replies using patterns learned from many examples of language and code.

Training repeats a prediction exercise on many examples:

1. Predict the next piece of text.
2. Compare the prediction with the example.
3. Adjust numbers inside the model.

The model learns patterns it can use in new requests.
Its learned numbers are called parameters.

The text comes in pieces called tokens

Software splits text into tokens and converts them to numbers.
A token can be a word, part of a word, or punctuation.

Example sentence

The hotel is expensive.

One possible tokenization

"The" " hotel" " is" " expensive" "."

Each quoted piece is one token. Here, spaces are included
at the start of the following word.

This is an illustration. Different models can split the same text differently.

Building a reply, one piece at a time

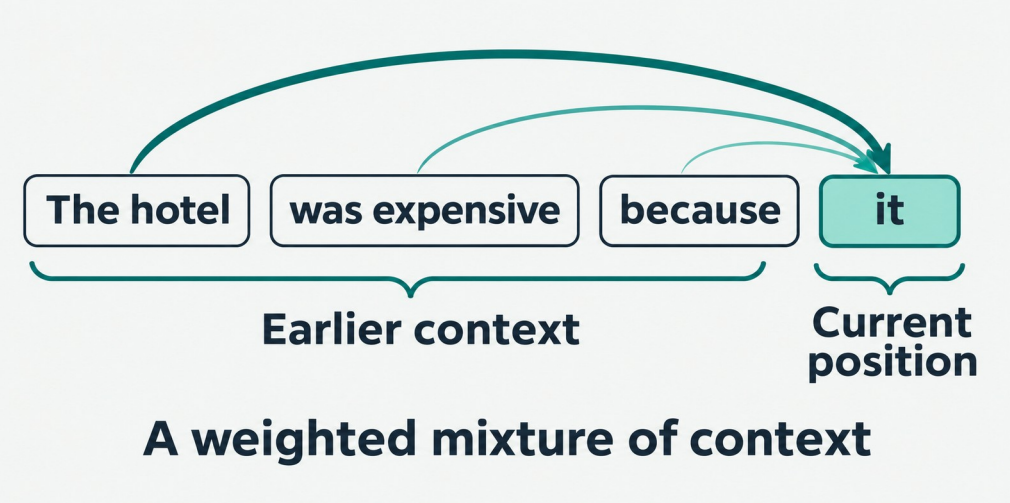
Text so far	`import pandas as`
One possible next piece	`" pd"`
Text after adding it	`import pandas as pd`

The model estimates a probability for each possible next token.
The system selects one, adds it to the text and repeats.

Illustrative sequence. Here the next piece includes a space. Token choices can vary.

Attention: which earlier words matter?

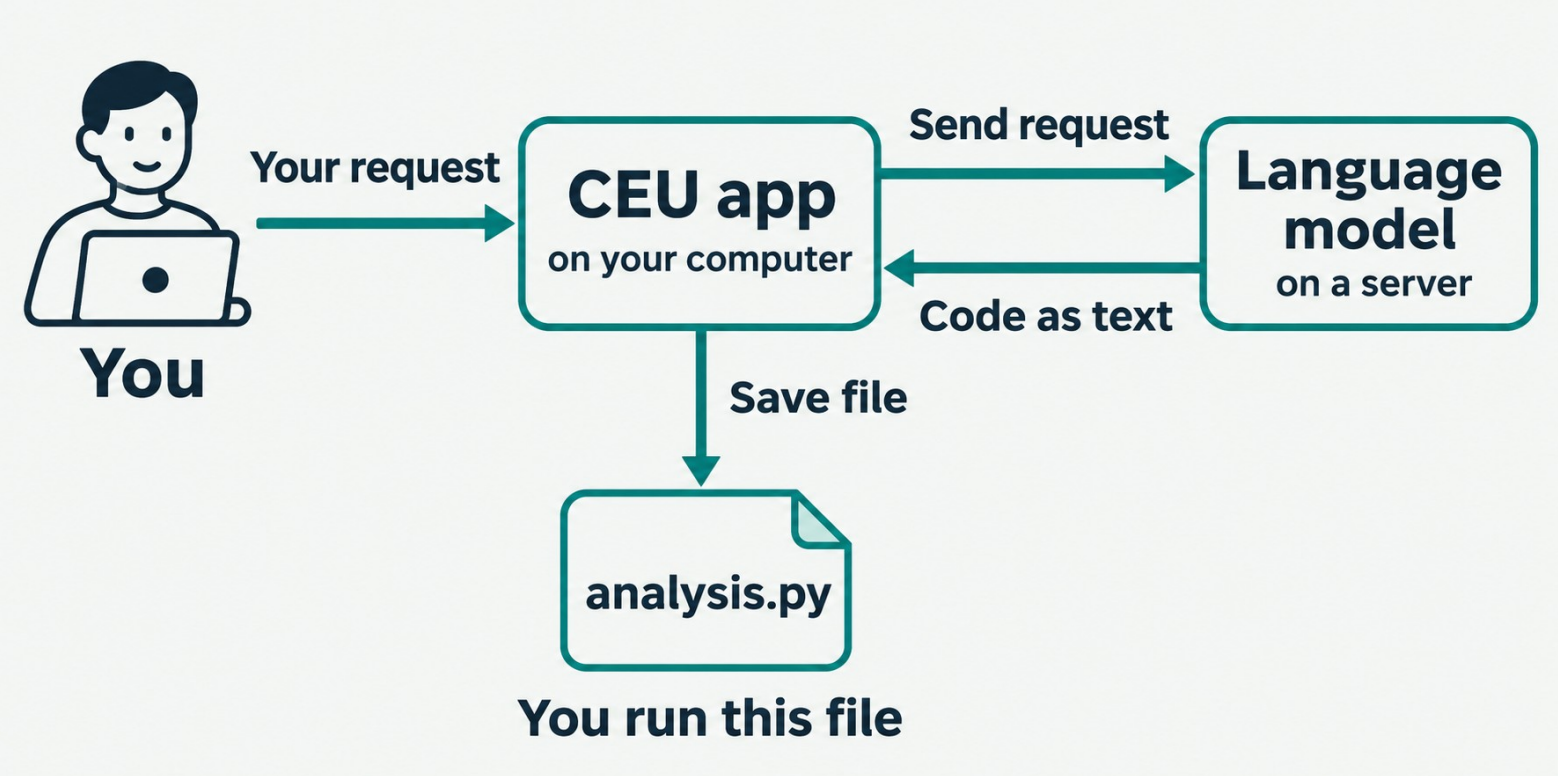
“The hotel was expensive because it was central.”
What does “it” refer to?



For “it”, the model combines information from earlier text pieces.
Some pieces contribute more than others. This operation is called attention.

The arrows illustrate the idea. They are not measured attention weights.

The app around the model



The software that connects the model to the user and handles its inputs and outputs is called a harness. Our ceu-ai.py app is a small harness.

The CEU API connects our app to the model

1. Our app sends an HTTPS request to the CEU API.
   The service passes the prompt and context to the selected model.

2. The model generates the reply.
   The CEU API returns it to our app in an HTTPS response.

HTTP defines rules for requests and responses.
HTTPS encrypts the connection.

The API specifies where to send the request and how to format it.
Our app handles those details for you.

Using our CEU app

From the course folder in the VS Code terminal:

```
uv run ceu-ai.py
```

First use: enter the class credentials. The app uses Gemini 3.1 Flash-Lite.
Then type your request, for example:

Create analysis.py. Load data/raw/hotels_course.csv
and print the number of rows and columns.

Open the saved file, inspect the code, then run it and check the result.

FirstRung: help with understanding

FirstRung asks questions about your code and offers support as you work out the answer.



For the cell from our example:

```
print(hotels.shape)
```

An example coaching question:
“What do the two numbers represent?”

Select the cell and choose “FirstRung: Coach me on this cell”.
Open the coaching panel, explain your answer, and ask for a hint if needed.

How we use these tools in Coding 2

We first write the analysis together in class.

At the end, each student writes a prompt asking CEU AI to recreate the analysis in a new file.

You run both versions and compare the data choices, the code and the results. You explain any differences.

FirstRung focuses on your understanding of the code.
You remain responsible for every result you submit.

What goes to the services

The CEU app sends prompts, conversation, course guidance, the folder tree and dataset schemas. It also sends files it reads or creates in the chat. CSV reads send summaries and small previews, not the full dataset.

FirstRung sends the selected work, available output and surrounding notebook text. In a course programme, it also reports estimates of time spent working on covered files.

Keep class credentials and personal data out of prompts and shared files.